딥러닝 프로그래밍

한국폴리텍대학 하이테크과정
2026년 2학기
박병주 강사

목차

A table of Contents

01 지난시간 복습

02 Learning Rate

03 Batch Size와 Epoch

04 Optimizer와 Weight Decay

05 LR Scheduler

06 튜닝 절차와 실험 5

PART 01

지난시간 복습

1. 복습: 실험 기록표

Baseline	없음, MLP 784-100-10	97.25% (10)	97.34%	기준
실험 1	optimizer 설정, SGD lr 0.1 → Adam lr 0.001	97.43% (10)	97.59%	+0.18%p
실험 2	모델, MLP → CNN	98.54% (9)	98.67%	+1.29%p
실험 4	모델 CNNAll(BN, Dropout), 증강	98.77% (10)	98.88%	+1.52%p

- 지금까지는 모델과 데이터를 바꿨고, 학습 조건(SGD lr 0.1, batch 64, 10 epoch)은 실험 1을 빼면 그대로 썼다
- 오늘은 이 학습 조건을 실험으로 골라 실험 5로 기록한다

1. 파라미터와 하이퍼파라미터

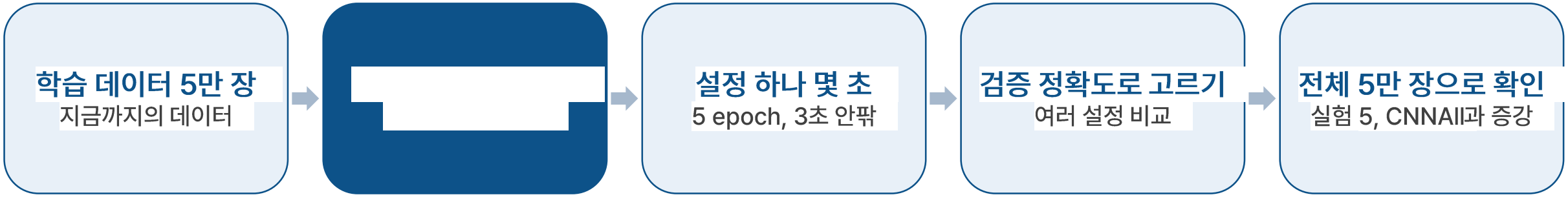
하이퍼파라미터
사람이 학습 전에 정한다
lr, batch size, epoch 수
optimizer, weight decay, scheduler

학습 방식을 정하면
학습이 값을 찾는다

파라미터
학습이 정한다 (opt.step())
weight와 bias
CNN은 20,490개

- 하이퍼파라미터는 학습 전에 사람이 정하는 값이고, 좋은 값은 계산이 아니라 실험으로 고른다 (튜닝)
- 층 수와 채널 수 같은 구조 값도 하이퍼파라미터다, 다음 시간에 다룬다

1. 튜닝 실험실



- 매 epoch 사진을 Tensor로 다시 바꾸지 않고, 한 번 바꿔 모아 둔 **1만 장**을 꺼내 쓴다
- 실험실에서 고른 설정은 **전체 데이터로 다시 확인**한다, 실험실의 1등이 늘 큰 실험의 1등은 아니다

1. 준비 셀 1: 데이터

준비 셀 1. 데이터와 튜닝 실험실

```
import torch
import torch.nn as nn
from torchvision import datasets, transforms
from torch.utils.data import DataLoader, random_split, Subset, TensorDataset
device = "cuda" if torch.cuda.is_available() else "cpu"
torch.backends.cudnn.deterministic = True       # cuDNN도 결정적 계산만
train_all = datasets.MNIST(root="data", train=True, download=True,
                           transform=transforms.ToTensor())
g = torch.Generator().manual_seed(0)            # 학습과 검증 분리, seed 0
train_set, val_set = random_split(train_all, [50000, 10000], generator=g)
lab = Subset(train_all, train_set.indices[:10000])   # 새 줄, 튜닝 실험실 1만 장
xs = torch.stack([img for img, _ in lab])            # 사진 1만 장을 한 텐서로
ys = torch.tensor([y for _, y in lab])
xv = torch.stack([img for img, _ in val_set])        # 검증 사진 1만 장도
yv = torch.tensor([y for _, y in val_set])
lab_set = TensorDataset(xs, ys)
val_fast = DataLoader(TensorDataset(xv, yv), batch_size=256, shuffle=False)
print(xs.shape, len(val_fast))   # → torch.Size([10000, 1, 28, 28]) 40
```

- 새 줄
  - import 줄의 `TensorDataset`과 아래쪽 일곱 줄
  - 사진 1만 장을 Tensor 하나로 쌓아 둔다
  - `TensorDataset`이 같은 번호의 사진과 정답을 짝짓는다
- 확인
  - `(10000, 1, 28, 28)`과 `40`이 나오면 성공이다

1. 준비 셀 2: CNN과 loss

준비 셀 2. 지난시간 CNN

```
class CNN(nn.Module):                          # 지난시간 CNN 그대로
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(1, 16, kernel_size=3, padding=1)
        self.conv2 = nn.Conv2d(16, 32, kernel_size=3, padding=1)
        self.pool = nn.MaxPool2d(2)
        self.act = nn.ReLU()
        self.flat = nn.Flatten()
        self.fc = nn.Linear(32 * 7 * 7, 10)
    def forward(self, x):
        x = self.pool(self.act(self.conv1(x)))   # (N, 16, 14, 14)
        x = self.pool(self.act(self.conv2(x)))   # (N, 32, 7, 7)
        return self.fc(self.flat(x))

loss_fn = nn.CrossEntropyLoss()
print(sum(p.numel() for p in CNN().parameters()))   # → 20490
```

- 지난시간 CNN 그대로이고, 파라미터는 20,490개다
- 비교할 때는 모델을 고정하고 학습 조건만 바꾼다

1. 준비 셀 3: 검증 함수와 학습 함수

준비 셀 3-1. 검증 함수

```python
def evaluate(model, loader):        # 검증 함수, 그대로
    model.eval()
    total_loss, correct = 0.0, 0
    with torch.no_grad():
        for x, y in loader:
            x, y = x.to(device), y.to(device)
            out = model(x)
            total_loss += loss_fn(out, y).item() * len(y)
            correct += (out.argmax(dim=1) == y).sum().item()
    n = len(loader.dataset)
    return total_loss / n, correct / n * 100
```

준비 셀 3-2. 학습 함수

```python
def train_one_epoch(model, loader, opt):    # 학습 함수, 그대로
    model.train()
    total_loss = 0.0
    for x, y in loader:
        x, y = x.to(device), y.to(device)
        loss = loss_fn(model(x), y)
        opt.zero_grad()
        loss.backward()
        opt.step()
        total_loss += loss.item() * len(y)
    return total_loss / len(loader.dataset)
```

- 두 함수는 그대로다, 학습 함수 안의 `opt.step()`이 오늘 바꿀 **optimizer와 lr**로 파라미터를 고친다

1. 준비 셀 4: fit, scheduler 자리 추가

준비 셀 4. 학습과 기록을 함수 하나로

```
def fit(model, loader, epochs=5, opt=None, sched=None):   # sched 자리 추가
    if opt is None:                      # 따로 주지 않으면 SGD lr 0.1
        opt = torch.optim.SGD(model.parameters(), lr=0.1)
    train_losses, val_losses, val_accs = [], [], []
    for epoch in range(epochs):
        train_losses.append(train_one_epoch(model, loader, opt))
        vl, va = evaluate(model, val_fast)       # 검증도 빠른 로더로
        val_losses.append(vl)
        val_accs.append(va)
        if sched is not None:            # 새 줄, epoch이 끝나면 lr 조정
            sched.step()
    best = max(val_accs)
    ep = val_accs.index(best) + 1        # best가 몇 번째 epoch인지
    print("best", ep, round(best, 2), "last", round(val_accs[-1], 2))
    return train_losses, val_losses, val_accs
```

- 바뀐 곳
  - 검증은 빠른 로더(`val_fast`)로 한다
  - `sched` 자리가 생겼다, 주지 않으면 아무 일도 하지 않는다
  - 기본 epoch은 5 (지난시간 50)
- 출력
  - best와 마지막 epoch의 검증 정확도를 함께 보여 준다

1. 예제 7-1: 빠른 로더 확인

예제 7-1. 지난시간 1,000장 실험을 빠른 로더로

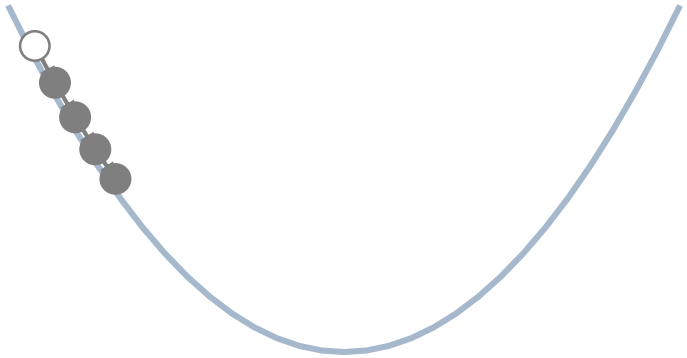
```
small_set = TensorDataset(xs[:1000], ys[:1000])   # 지난시간 1,000장과 같은 사진
small_loader = DataLoader(small_set, batch_size=64, shuffle=True)
torch.manual_seed(0)
model = CNN().to(device)
train_losses, val_losses, val_accs = fit(model, small_loader, 50)
# → (예) best 49 93.88 last 93.73   (지난시간 결과와 같다)
```

- 지난시간 1,000장 실험과 같은 best 49 epoch 93.88%, 마지막 93.73%가 나온다(같은 GPU 환경), 로더만 빨라졌고 결과는 같다
- 실험실의 실습과 비교는 이 빠른 로더로 한다

PART 02

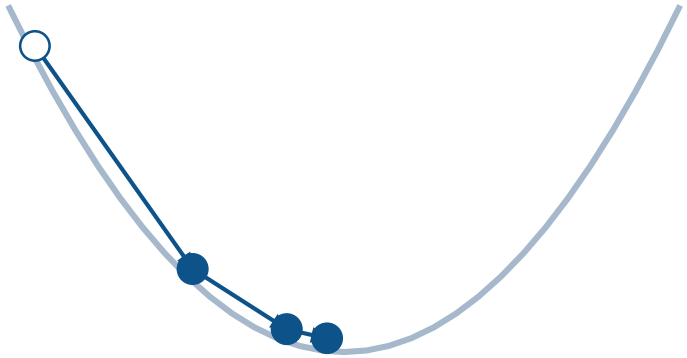
Learning Rate

2. 학습률이 하는 일



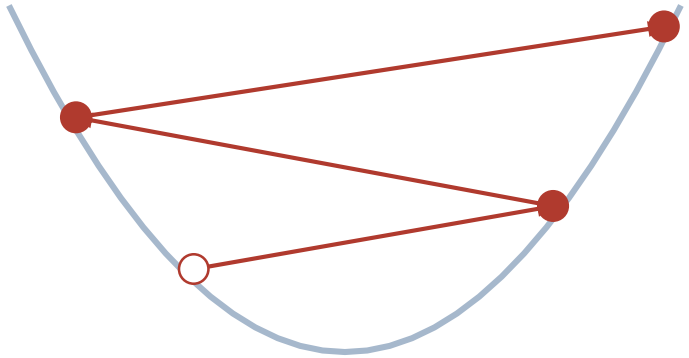
너무 작다 (lr 0.001)

걸음이 짧아 5 epoch 마지막 38.53%
바닥까지 한참 남았다



알맞다 (lr 0.1)

몇 걸음 만에 바닥 근처로
5 epoch 마지막 96.35%



너무 크다 (lr 1.0)

처음 몇 걸음에 크게 튀어 망가진다
마지막 9.89%, 한 숫자만 답한다

파라미터 갱신 $\theta \leftarrow \theta - \eta \frac{\partial L}{\partial \theta}$ θ(세타)는 파라미터, L은 loss, ∂L/∂θ는 기울기, η(에타)가 lr, ←는 왼쪽 값을 오른쪽 계산 결과로 바꾼다는 뜻

- lr은 한 번의 갱신(걸음)에서 기울기 반대 방향으로 **얼마나 옮길지**를 정한다, 너무 작으면 느리고 너무 크면 튄다

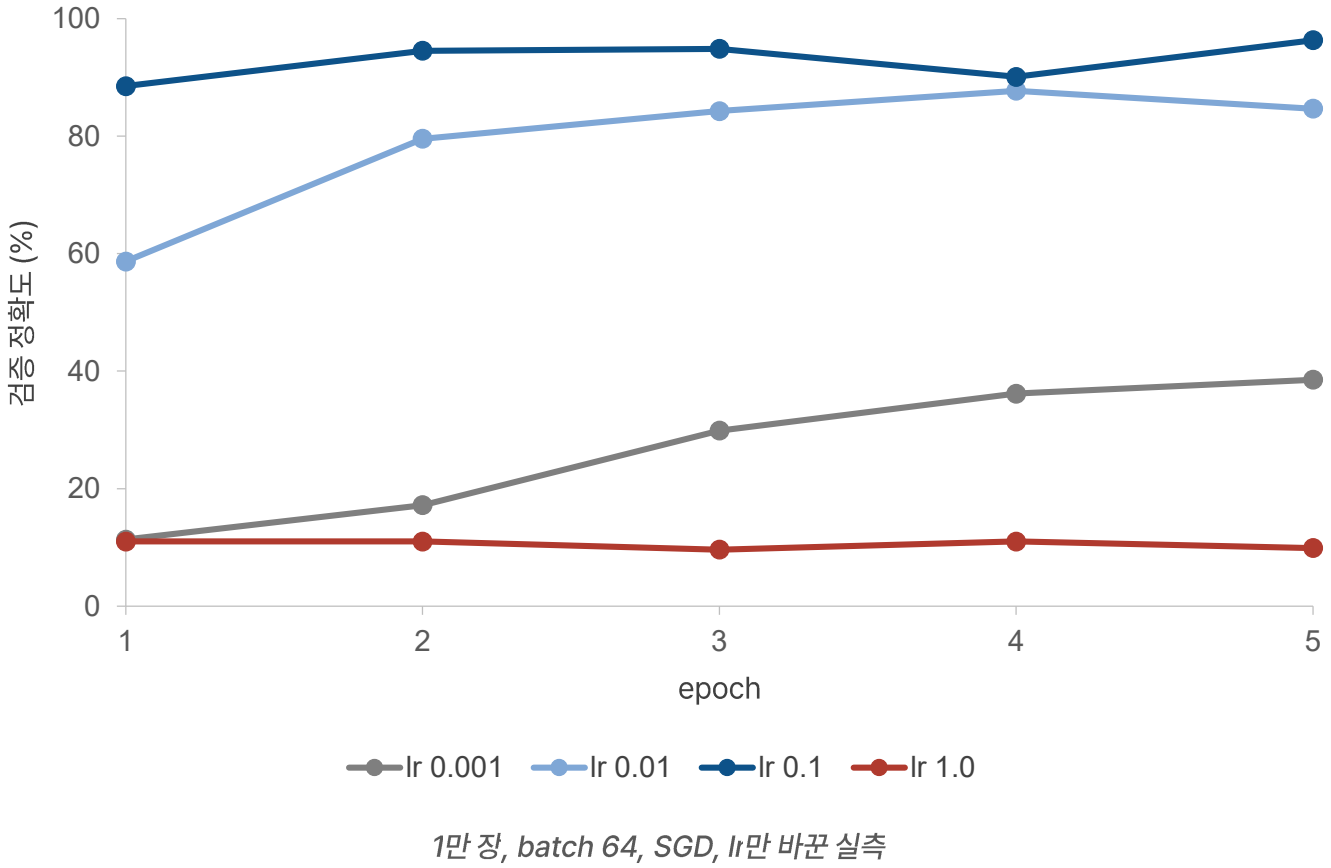
2. 예제 7-2: lr 네 개 비교

예제 7-2. 같은 조건, lr만 바꿔 5 epoch

```
lab_loader = DataLoader(lab_set, batch_size=64, shuffle=True)
results = {}                                  # lr마다 기록을 모아 둔다
for lr in (0.001, 0.01, 0.1, 1.0):
    torch.manual_seed(0)                      # 모두 같은 출발점
    model = CNN().to(device)
    opt = torch.optim.SGD(model.parameters(), lr=lr)
    print("lr", lr, end=" ")
    results[lr] = fit(model, lab_loader, 5, opt)
# → (예) lr 0.001 best 5 38.53 last 38.53
# → (예) lr 0.01 best 4 87.72 last 84.67
# → (예) lr 0.1 best 5 96.35 last 96.35
# → (예) lr 1.0 best 1 10.99 last 9.89
```

- 0.1이 96.35%로 가장 좋고, 0.001은 38.53%로 느리며, 1.0은 best도 10.99%로 찍는 수준이다

2. 결과: lr에 따른 학습 곡선



1만 장, batch 64, SGD, lr만 바꾼 실측

- 0.001은 내내 조금씩 오르고, 0.01은 느리며, 0.1은 2 epoch에 90%를 넘는다
- 1.0은 처음부터 10% 근처에 갇혔다
- 0.1도 4 epoch에 90.09%로 잠깐 떨어진다, lr이 크면 흔들림도 크다

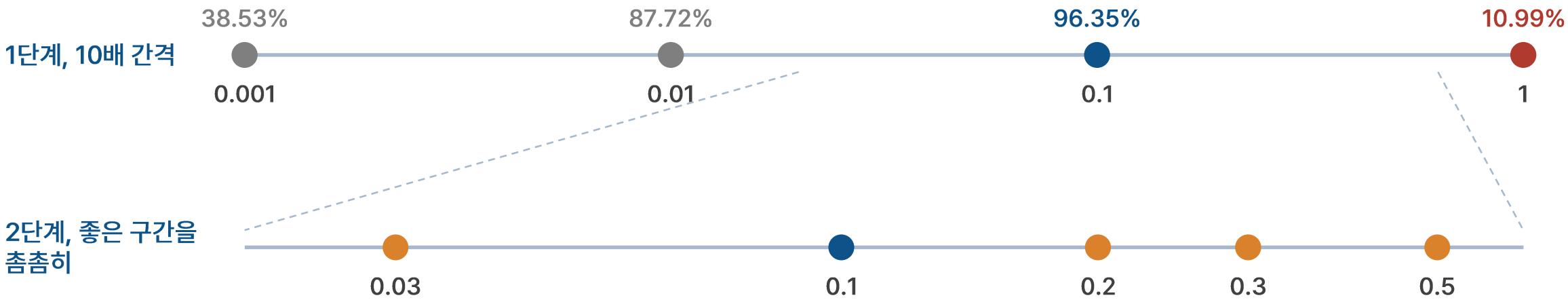
2. lr이 너무 크면

1.0	2.303 → 2.304 → 2.305 → 2.305 → 2.304	9.89%	너무 크다, 줄지 않고 한 숫자만 답한다
0.001	2.303 → 2.292 → 2.281 → 2.269 → 2.255	38.53%	너무 작다, 줄고는 있지만 느리다
0.1	0.804 → 0.216 → 0.157 → 0.126 → 0.107	96.35%	알맞다

찍는 수준의 loss $L = -\ln\frac{1}{10} \approx 2.303$ L은 loss, ln은 자연로그(log), 10개 숫자에 고르게 0.1씩 확률을 주면 loss는 2.303이다

- 모델과 데이터가 같을 때, loss가 2.30 근처에서 **줄지 않고 오르내리거나 커지면** 너무 크다는 신호, **아주 조금씩 꾸준히** 줄면 너무 작다는 신호

2. lr을 고르는 순서



1단계 숫자는 예제 7-2의 best 검증 정확도, 2단계 주황 점은 실습 1에서 시험할 값(파랑 0.1은 비교 기준)

- 처음에는 **10배 간격**으로 넓게 보고, 좋은 구간을 찾으면 그 안을 **촘촘히** 본다
- best만이 아니라 **마지막 값**도 본다, best가 가장 좋은 lr의 best와 비슷한데 마지막 값이 떨어지면 lr이 큰 쪽이다

2. 실습 1: lr 범위 좁히기 (문제)

실습

1. 예제 7-2를 복사해 lr을 0.03, 0.3, 0.5로 바꿔 학습하기
2. 일곱 개 lr의 best와 마지막 검증 정확도를 표에 적고, 가장 좋은 구간 찾기
3. 0.2처럼 0.1과 0.3 사이의 lr로 한 번 더 좁혀 보기

best	38.53	87.72		96.35			10.99
마지막	38.53	84.67		96.35			9.89

- 예제 7-2의 for 줄부터 복사한다, `results = {}`까지 복사하면 앞 기록이 지워진다
- 3은 for 줄의 목록을 `[0.2]`로 바꾼다
- 같은 seed, 같은 1만 장, 같은 5 epoch에서 lr만 바꾼다

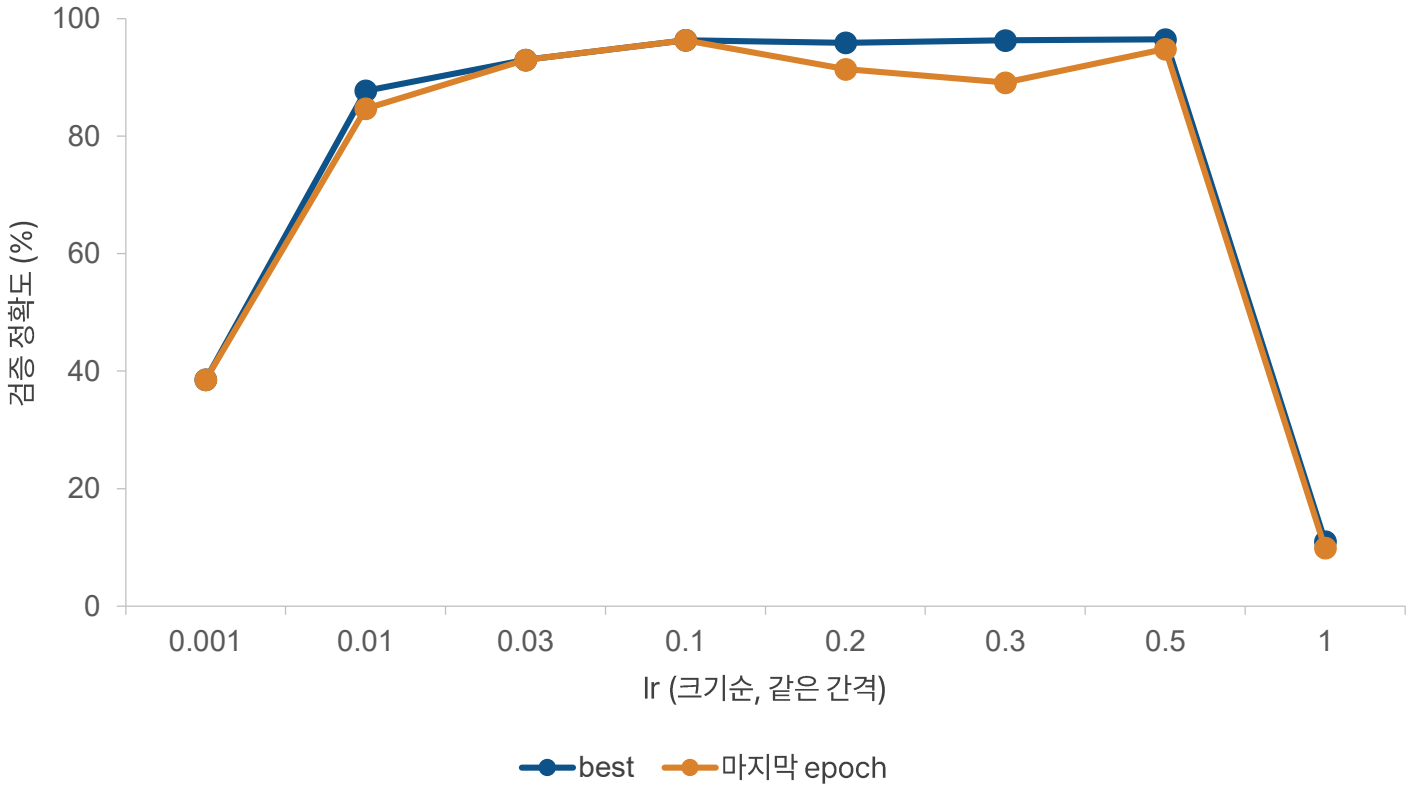
2. 실습 1: 정답 예시

실습 1. 정답 예시

```
for lr in (0.03, 0.3, 0.5):                    # 1. 0.01과 1.0 사이를 좁힌다
    torch.manual_seed(0)
    model = CNN().to(device)
    opt = torch.optim.SGD(model.parameters(), lr=lr)
    print("lr", lr, end=" ")
    results[lr] = fit(model, lab_loader, 5, opt)
# → (예) lr 0.03 best 5 93.0 last 93.0
# → (예) lr 0.3 best 3 96.3 last 89.11
# → (예) lr 0.5 best 2 96.47 last 94.84
```

- 1. 세 lr의 best / 마지막 (%)
  - 0.03은 93.00 / 93.00, 0.3은 96.30 / 89.11, 0.5는 96.47 / 94.84
- 2. 가장 좋은 구간
  - best로는 0.1~0.5가 비슷하고, 마지막 값까지 보면 0.1 근처가 낫다
- 3. 더 좁히기
  - 0.2도 마지막 91.40%로 0.1보다 낫지 않았다

2. 결과: lr과 검증 정확도



1만 장, 5 epoch, SGD, batch 64 실측

- 너무 작으면 덜 배우고, 너무 크면 흔들리다 무너진다
- 좋은 lr은 꼭대기 하나가 아니라 구간이다, 그 안에서 안정적인 값을 고른다

2. 예제 문제, lr 진단

문제

1. 처음 보는 lr A의 5 epoch train loss 기록(2.307 → 2.303 → 2.300 → 2.297 → 2.294), lr에 무슨 문제가 있을까?
2. 다른 lr B의 기록(2.319 → 2.313 → 2.315 → 2.315 → 2.313), lr에 무슨 문제가 있을까?
3. lr 0.1, 같은 설정에서 seed만 1로 바꾸면 5 epoch 마지막 값이 96.35%와 같을까?

2. 예제 문제 풀이

1	너무 작다, A는 lr 0.0003	꾸준히 아주 조금씩 준다, 검증 13.10%
2	너무 크다, B는 lr 3.0	줄지 않고 검증 9.92%, 한 숫자만 답한다
3	아니다, seed가 바뀌면 결과도 조금 다르다	seed 1은 96.77%

- lr은 가장 먼저 고르는 하이퍼파라미터다, 다른 값을 바꾸면 **lr도 다시** 본다

PART 03

Batch Size와 Epoch

3. batch size가 바꾸는 것

epoch당 갱신 횟수 $S = \left\lceil \frac{N}{B} \right\rceil$

S는 epoch당 갱신 횟수, N은 학습 사진 수(실험실은 1만 장), B는 batch size, [ ]는 소수점을 올린다는 뜻(마지막 덜 찬 배치도 한 번)

16	625	16장	자주 고친다, 오래 걸린다
64	157	64장	지금까지 쓴 값
256	40	256장	빨리 끝난다, 고치는 횟수가 적다
1024	10	1024장	아주 빠르지만 5 epoch에 50번만 고친다

- batch가 크면 epoch은 빨리 끝나지만 고치는 횟수가 줄어든다

3. 예제 7-3: batch size 네 개 비교

예제 7-3. 같은 lr 0.1, batch만 바꿔 5 epoch

```
import time
for bs in (16, 64, 256, 1024):
    loader = DataLoader(lab_set, batch_size=bs, shuffle=True)
    torch.manual_seed(0)
    model = CNN().to(device)
    opt = torch.optim.SGD(model.parameters(), lr=0.1)  # lr 0.1 그대로
    t0 = time.time()
    print("batch", bs, "steps", len(loader), end=" ")   # epoch당 step 수
    fit(model, loader, 5, opt)
    print("    time", round(time.time() - t0, 1))       # 걸린 시간(초)
# → (예) batch 16 steps 625 best 5 97.29 last 97.29
# → (예)     time 11.8
# → (예) batch 64 steps 157 best 5 96.35 last 96.35
# → (예)     time 3.4
# → (예) batch 256 steps 40 best 3 90.33 last 87.31
# → (예)     time 1.0
# → (예) batch 1024 steps 10 best 4 80.6 last 78.25
# → (예)     time 1.0
```

- 읽는 법
  - `len(loader)`는 epoch당 배치 수, 곧 고치는 횟수다
  - lr 0.1은 `opt`로 직접 넘긴다
  - `time.time()`의 뺄셈이 걸린 시간(초)이다
- 결과
  - batch가 클수록 빨리 끝나지만 정확도는 떨어졌다

3. 결과: 정확도와 시간

16	625	97.29%	97.29%	11.8초
64	157	96.35%	96.35%	3.4초
256	40	90.33%	87.31%	1.0초
1024	10	80.60%	78.25%	1.0초

1만 장, SGD lr 0.1, 5 epoch 실측 (시간은 측정한 GPU 기준)

- batch 16은 가장 높지만 64보다 **약 3배** 오래 걸렸다 (시간은 실행마다 조금씩 다르다)
- lr을 0.1로 고정한 비교다, batch를 바꾸면 알맞은 lr도 다시 봐야 한다

3. 실습 2: batch를 키우면 lr도 (문제)

실습

1. batch 256, lr 0.4(64일 때의 4배)로 5 epoch 학습하기
2. 예제 7-3의 batch 256(lr 0.1, best 90.33%)과 비교하기
3. batch 1024에 lr 1.6으로 학습하고 결과를 설명하기

선형 스케일링 $$\eta_{\text{new}} = \eta \times \frac{B_{\text{new}}}{B}$$ B는 원래 batch, η는 원래 lr, B_{new}와 η_{new}는 바꾼 값, 64 → 256이면 0.1 → 0.4

- 예제 7-3을 복사해 for 줄의 batch 목록을 `[256]`으로, `SGD`의 lr을 0.4로 바꾼다
- 3은 batch 목록을 `[1024]`로, lr을 1.6으로 바꾼다

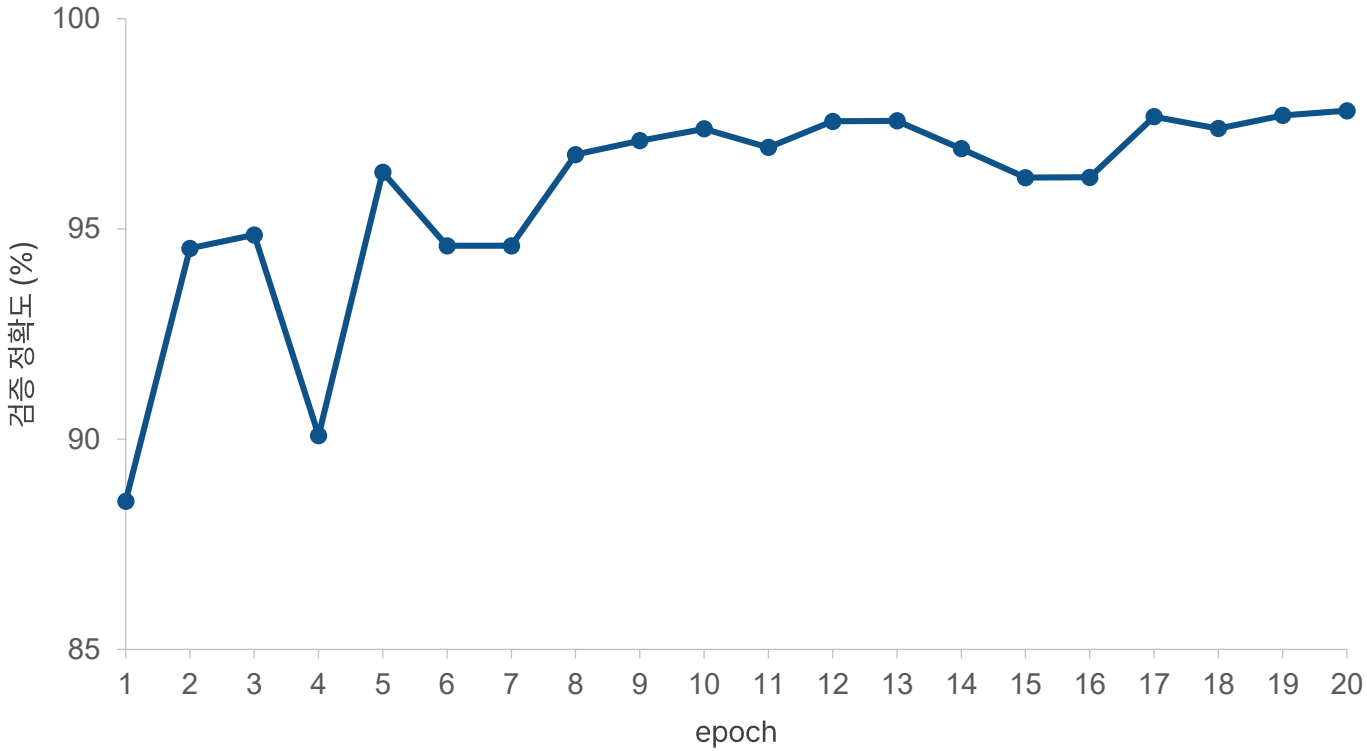
3. 실습 2: 정답 예시

실습 2. 정답 예시

```
import time
for bs in [256]:                              # 1. batch는 256 하나 (3은 [1024])
    loader = DataLoader(lab_set, batch_size=bs, shuffle=True)
    torch.manual_seed(0)
    model = CNN().to(device)
    opt = torch.optim.SGD(model.parameters(), lr=0.4)  # lr도 4배 (3은 1.6)
    t0 = time.time()
    print("batch", bs, "steps", len(loader), end=" ")
    fit(model, loader, 5, opt)
    print("    time", round(time.time() - t0, 1))
# → (예) batch 256 steps 40 best 3 93.42 last 92.26
# → (예)     time 1.2
```

- 1. batch 256, lr 0.4
  - best 93.42%
- 2. 비교
  - lr 0.1의 90.33%보다 높다, 고치는 횟수가 준 만큼 lr을 키운 효과
- 3. batch 1024, lr 1.6
  - best 19.10%, 마지막 10.99%로 무너졌다, 마지막은 한 숫자만 답하는 상태
  - 규칙은 출발점일 뿐, batch를 바꾸면 lr을 다시 본다

3. 예제 7-4: epoch 수는 검증 곡선으로



예제 7-4, 1만 징, SGD lr 0.1, 20 epoch 실측

- 이 seed에서는 처음 몇 epoch에 크게 오르고, 10 epoch 이후는 조금씩 오르내린다
- epoch 수는 검증 곡선이 평평해지는 곳으로 고르거나 Early Stopping으로 정한다
- 실험실은 비교를 빨리 하려고 5 epoch만 쓴다

3. 예제 문제, batch와 epoch

문제

1. batch를 64에서 1024로 바꾸고 lr과 epoch은 그대로 뒀더니 96.35%가 80.60%로 떨어졌다, 왜일까?
2. batch 16이 97.29%로 가장 높았다, 그럼 늘 batch 16을 쓰는 것이 좋을까?
3. 5 epoch에서는 batch 16(97.29%)이 64(96.35%)를 이겼다, 20 epoch으로 길게 학습해도 그럴까?

3. 예제 문제 풀이

1	epoch당 고치는 횟수가 157번에서 10번으로 줄었다, 큰 batch는 lr을 다시 찾아야 한다	5 epoch에 785번 대 50번, lr을 16배로 키우자 오히려 19.10%
2	아니다, 시간도 함께 본다	batch 64보다 약 3배 오래 걸렸다
3	이번에는 그랬다, 다만 차이가 크게 줄었다	차이가 5 epoch 0.94%p에서 20 epoch 0.17%p로

- 짧은 비교의 차이는 길게 학습하면 **줄어들 수 있다**, 고른 설정은 마지막에 길게 다시 확인한다

PART 04

Optimizer와 Weight Decay

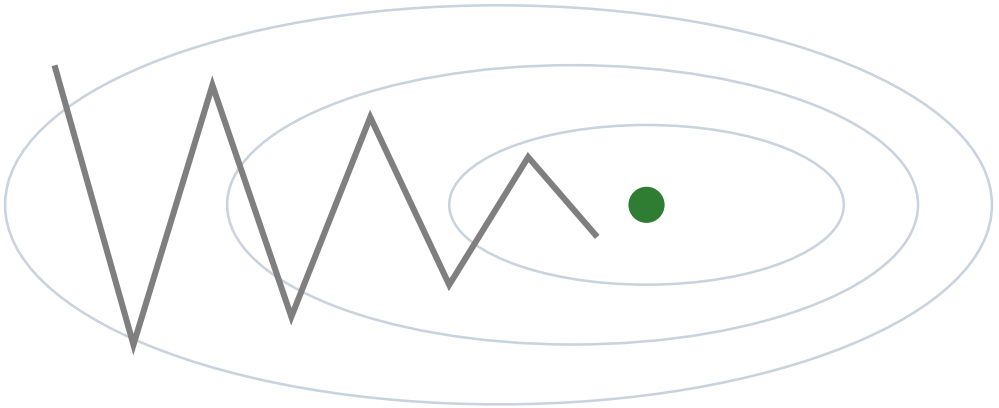
4. optimizer 네 가지

SGD	기울기에 lr을 곱한 만큼 반대 방향으로	0.1	`SGD(p, lr=0.1)`
SGD + momentum	지난 갱신 방향을 이어받아 부드럽게	0.01	`SGD(p, lr=0.01, momentum=0.9)`
Adam	파라미터마다 갱신 크기를 자동으로 맞춘다	0.001	`Adam(p, lr=0.001)`
AdamW	Adam + weight decay를 따로 적용	0.001	`AdamW(p, lr=0.001, weight_decay=0.01)`

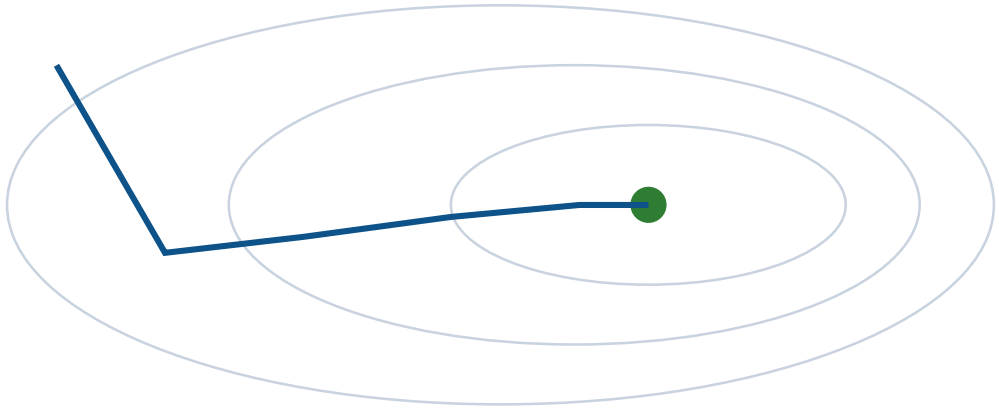
Adam의 갱신 $\theta \leftarrow \theta - \eta \frac{m}{\sqrt{s} + \varepsilon}$ θ, η는 갱신 식과 같고, m은 최근 기울기의 평균, s는 최근 기울기 제곱의 평균(둘 다 지난 값을 조금씩 잊는 평균, 처음 몇 번의 갱신은 PyTorch가 보정), ε은 0으로 나누지 않게 더하는 아주 작은 수

- 알맞은 lr 범위는 **optimizer마다 다르다, 바꾸면 lr도 다시 본다** (표의 p는 `model.parameters()`)

4. momentum의 원리



SGD, 지그재그로 느리게



momentum, 방향을 모아 부드럽게

초록 점이 loss가 가장 낮은 곳

momentum $v \leftarrow \mu v + \frac{\partial L}{\partial \theta}, \qquad \theta \leftarrow \theta - \eta v$

v는 기울기를 이어받아 쌓은 값(실제 갱신량은 η × v), μ(뮤, 보통 0.9)는 지난 v를 이어받는 비율, θ, η, ∂L/∂θ는 파라미터 갱신 식과 같다

- 같은 기울기가 계속되면 갱신량이 약 10배까지 커지므로 **lr을 낮춰** 시작한다

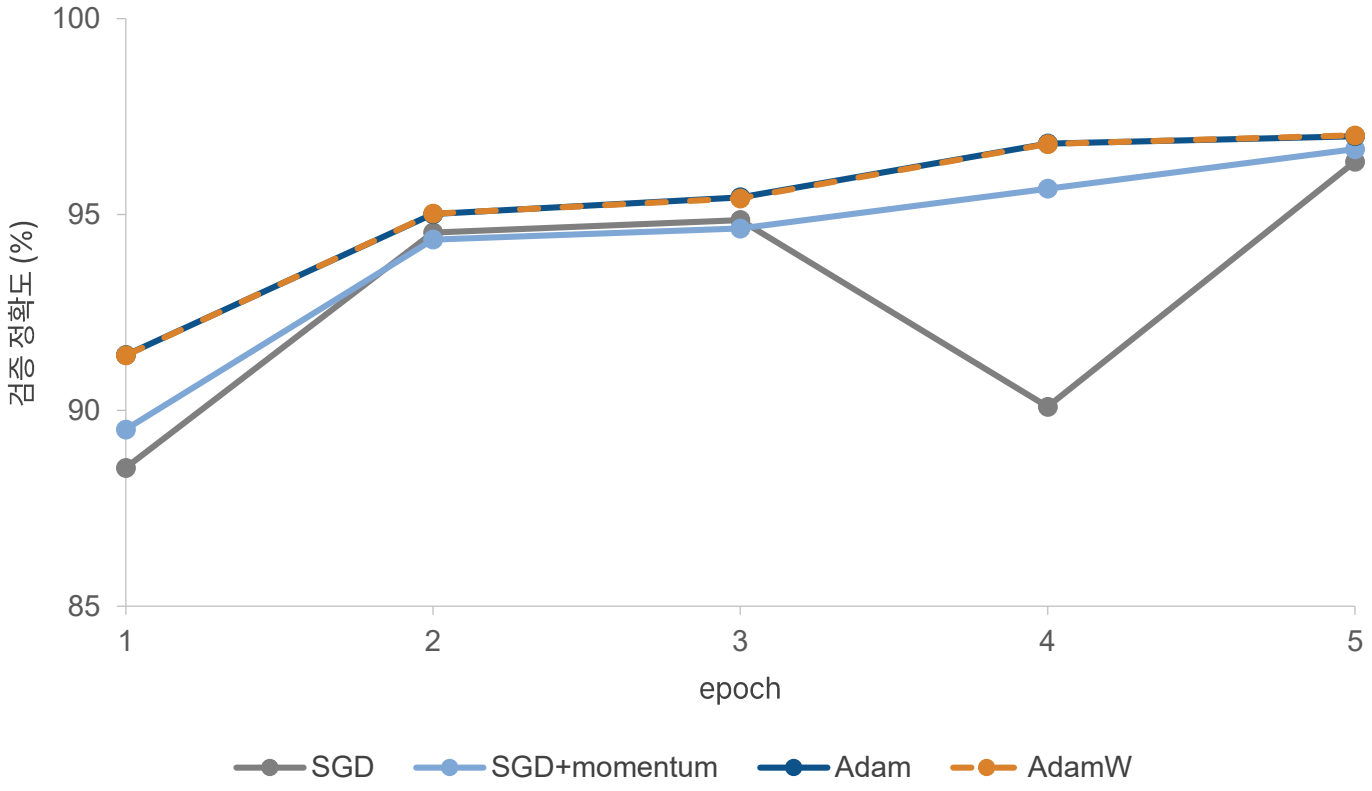
4. 예제 7-5: optimizer 네 개 비교

예제 7-5. 같은 CNN, optimizer를 각자의 흔한 출발 lr로 5 epoch

```
def make_opt(name, params):                # 이름으로 optimizer를 만든다
    if name == "SGD":
        return torch.optim.SGD(params, lr=0.1)
    if name == "SGD+momentum":
        return torch.optim.SGD(params, lr=0.01, momentum=0.9)
    if name == "Adam":
        return torch.optim.Adam(params, lr=0.001)
    return torch.optim.AdamW(params, lr=0.001, weight_decay=0.01)
opt_results = {}
for name in ("SGD", "SGD+momentum", "Adam", "AdamW"):
    torch.manual_seed(0)
    model = CNN().to(device)
    opt = make_opt(name, model.parameters())
    print(name, end=" ")
    opt_results[name] = fit(model, lab_loader, 5, opt)
# → (예) SGD best 5 96.35 last 96.35
# → (예) SGD+momentum best 5 96.67 last 96.67
# → (예) Adam best 5 97.0 last 97.0
# → (예) AdamW best 5 97.02 last 97.02
```

- 읽는 법
  - `make_opt`는 이름으로 optimizer를 만드는 함수다
  - 각자 흔한 출발 lr을 썼다
- 결과
  - 모두 96~97%대로 차이는 작다, 이번에는 Adam 계열이 조금 높았다

4. 결과: optimizer별 학습 곡선



1만 징, batch 64, 5 epoch 실측

- Adam 계열은 첫 epoch부터 높고 매끈하다, AdamW(점선)는 Adam과 거의 겹친다
- SGD는 4 epoch에 90.09%로 흔들렸고, momentum은 덜 흔들렸다
- 끝의 차이는 1%p 안쪽이라 seed 하나로 순위를 정하기 어렵다

4. weight decay

weight decay $$\theta \leftarrow \theta - \eta \left(\frac{\partial L}{\partial \theta} + \lambda \theta \right)$$

θ는 파라미터, η는 lr, ∂L/∂θ는 기울기, λ(람다)는 weight_decay 값, 한 번 갱신할 때 lr × λ × θ만큼 0 쪽으로 줄인다 (lr 0.001, λ 1.0이면 0.1%씩)

SGD(weight_decay=λ)	식 그대로, 기울기에 λθ를 더한다	모든 weight를 같은 비율로 줄인다
Adam(weight_decay=λ)	λθ를 더한 뒤 Adam이 갱신 크기를 다시 맞춘다	줄이는 양이 weight마다 달라진다
AdamW(weight_decay=λ)	Adam의 갱신과 따로 lr × λ × θ만큼 줄인다	모든 weight를 같은 비율로 줄인다

- weight decay는 weight가 지나치게 커지지 않게 매번 조금씩 줄인다, 과적합을 줄이려는 도구다
- Adam에 weight decay를 주려면 **AdamW**를 쓴다, 실측 비교는 실습 3 정답에서 본다

4. 실습 3: 1,000장에서 weight decay (문제)

실습

1. AdamW(lr 0.001)의 weight_decay를 0, 0.1, 1.0으로 바꿔 1,000장으로 50 epoch씩 학습하기
2. best 검증 정확도, 학습 데이터 정확도, weight 크기(`weight_size`)를 노트북의 표에 적기
3. weight decay가 무엇을 바꿨는지 한 문장으로 쓰기

weight 크기를 재는 함수

```
def weight_size(model):      # 제곱합의 루트
    total = 0.0
    for name, p in model.named_parameters():
        if "weight" in name:     # bias는 빼고
            total += (p ** 2).sum().item()
    return total ** 0.5
```

- 힌트
  - 예제 7-2의 for 줄부터 복사해 위에서부터 바꾼다
  - for 줄은 `for wd in (0.0, 0.1, 1.0):`
  - opt 줄은 `opt = torch.optim.AdamW(model.parameters(), lr=0.001, weight_decay=wd)`
  - print 줄은 `print("wd", wd, end=" ")`, results 줄은 `fit(model, small_loader, 50, opt)`로 바꾼다
  - fit 줄 바로 아래, 같은 깊이에 `print(round(evaluate(model, small_loader)[1], 2), round(weight_size(model), 2))`

4. 실습 3: 정답 예시

실습 3. 정답 예시

```
for wd in (0.0, 0.1, 1.0):                    # 1. weight decay 세 가지
    torch.manual_seed(0)
    model = CNN().to(device)
    opt = torch.optim.AdamW(model.parameters(), lr=0.001, weight_decay=wd)
    print("wd", wd, end=" ")
    fit(model, small_loader, 50, opt)                  # 1,000장, 50 epoch
    # 2. 학습 데이터 정확도와 weight 크기
    print(round(evaluate(model, small_loader)[1], 2), round(weight_size(model), 2))
# → (예) wd 0.0 best 36 93.26 last 93.16
# → (예) 100.0 9.56
# → (예) wd 0.1 best 46 93.31 last 93.27
# → (예) 100.0 9.23
# → (예) wd 1.0 best 48 93.68 last 93.54
# → (예) 100.0 7.34
```

- 1, 2. 표에 적을 값
  - weight 크기 9.56 → 9.23 → 7.34로 줄었다
  - best 93.26 → 93.31 → 93.68%, 학습 데이터는 100%
- 3. 한 문장 정리
  - weight를 작게 유지한다, 정확도 효과는 +0.42%p로 작다
- weight_decay 0.1을 Adam에 주면
  - weight가 3.12까지 지나치게 줄고 best도 90.42%로 낮다 (AdamW는 9.23, 93.31%)

4. 예제 문제, optimizer 진단

문제

1. SGD에서 잘 되던 lr 0.1을 Adam에 그대로 주면 어떻게 될까?
2. SGD lr 0.1은 그대로 두고 momentum 0.9만 더하면 어떻게 될까?
3. weight decay 1.0으로 weight 크기가 9.56에서 7.34로 줄었다, 정확도도 반드시 크게 오를까?

4. 예제 문제 풀이

1	무너진다, Adam에는 너무 큰 lr이다	Adam lr 0.1은 best 10.99%
2	이 실험실에서는 97.05%로 오히려 좋았다, 다만 갱신량이 커져 흔들릴 수 있다	SGD만 쓰면 96.35%
3	아니다, 효과는 있지만 작다	seed 세 번 모두 0.19~0.42%p 높았다

- optimizer를 바꾸면 lr 범위부터 다시 본다

PART 05

LR Scheduler

5. scheduler의 원리

StepLR

$$\eta_t = \eta_0 \, \gamma^{\lfloor t/k \rfloor}$$

η_t는 t epoch이 지난 뒤의 lr, η_0은 처음 lr, $\lfloor\ \rfloor$는 소수점 버림, k(step_size) epoch마다 γ(gamma)를 곱한다

Cosine

$$\eta_t = \frac{\eta_0}{2}\left(1 + \cos\frac{\pi t}{T}\right)$$

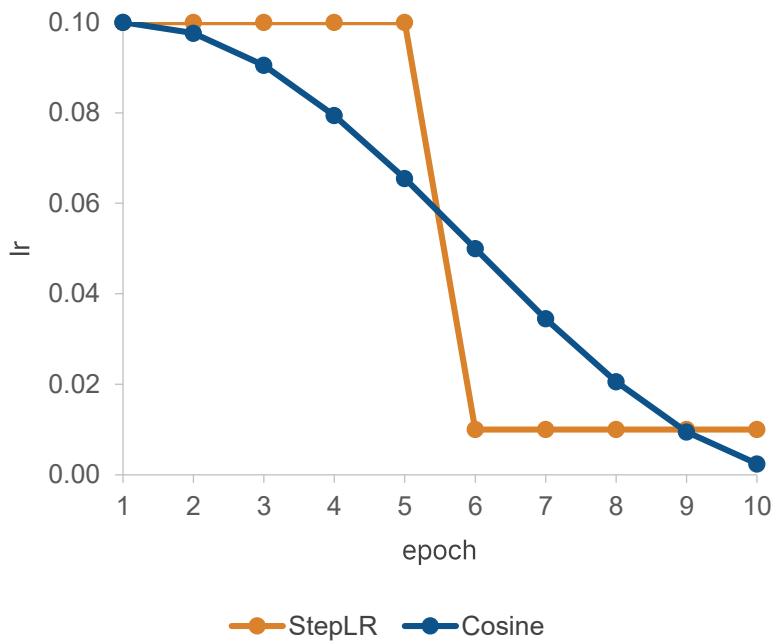
T(T_max) epoch 동안 코사인 곡선을 따라 0까지 부드럽게 줄인다, π는 원주율로 t = T이면 cos π = −1이라 lr은 0 (lr이 내려가는 바닥값 eta_min이 기본 0일 때의 식)

- 처음에는 **큰 lr**로 빨리 내려가고, 끝에는 **작은 lr**로 자리를 잡는다
- StepLR(step_size=5, gamma=0.1)이면 5 epoch마다 lr이 **10분의 1**이 된다
- `sched.step()`은 **epoch이 끝날 때** 한 번 부른다

5. 예제 7-6: lr이 바뀌는 모양

예제 7-6. StepLR과 Cosine, 10 epoch 동안의 lr

```
w = torch.zeros(1, requires_grad=True)      # lr만 보려는 가짜 파라미터
for kind in ("step", "cos"):
    opt = torch.optim.SGD([w], lr=0.1)
    if kind == "step":
        sched = torch.optim.lr_scheduler.StepLR(opt, step_size=5, gamma=0.1)
    else:
        sched = torch.optim.lr_scheduler.CosineAnnealingLR(opt, T_max=10)
    lrs = []
    for epoch in range(10):
        lrs.append(round(opt.param_groups[0]["lr"], 4))    # 이번 epoch의 lr
        opt.step()                                          # (학습 자리)
        sched.step()                                        # epoch이 끝나면 조정
    print(kind, lrs)
# → step [0.1, 0.1, 0.1, 0.1, 0.1, 0.01, 0.01, 0.01, 0.01, 0.01]
# → cos [0.1, 0.0976, 0.0905, 0.0794, 0.0655, 0.05, 0.0345, 0.0206, 0.0095, 0.0024]
```



- StepLR은 계단처럼, Cosine은 부드럽게 줄어든다

5. 예제 7-7: scheduler로 학습

예제 7-7. 같은 lr 0.1, 상수 lr과 Cosine을 10 epoch

```
sched_results = {}                          # 방식마다 기록을 모아 둔다
for kind in ("const", "cos"):
    torch.manual_seed(0)
    model = CNN().to(device)
    opt = torch.optim.SGD(model.parameters(), lr=0.1)
    sched = None                                  # const는 lr 0.1 그대로
    if kind == "cos":
        sched = torch.optim.lr_scheduler.CosineAnnealingLR(opt, T_max=10)
    print(kind, end=" ")
    sched_results[kind] = fit(model, lab_loader, 10, opt, sched)
# → (예) const best 10 97.38 last 97.38
# → (예) cos best 9 97.11 last 97.1
```

- seed 0에서는 상수 lr이 마지막 97.38%로 Cosine(97.10%)보다 조금 높았다

5. 실습 4: StepLR로 학습 (문제)

실습

1. 예제 7-7을 복사해 방식에 StepLR(step_size=5, gamma=0.1)을 더해 10 epoch 학습하기
2. 상수 lr, Cosine과 마지막 검증 정확도를 비교하기
3. step_size를 3으로 바꾸면 lr이 어떻게 바뀌는지 예제 7-6처럼 출력해 보기

- 힌트
  - for 줄의 방식 목록에 `"step"`을 더한다, `("const", "cos", "step")`
  - 아래 상자처럼 cos의 if 두 줄 바로 아래에 넣는다, if 줄은 cos의 if와 같은 깊이, sched 줄은 한 칸 더
  - 3은 예제 7-6의 `step_size=5`를 3으로 바꿔 다시 실행한다

새 두 줄을 넣는 자리 (위아래는 예제 7-7 그대로)

```
    if kind == "cos":
        sched = torch.optim.lr_scheduler.CosineAnnealingLR(opt, T_max=10)
    if kind == "step":                      # 새 두 줄, cos의 if와 같은 깊이
        sched = torch.optim.lr_scheduler.StepLR(opt, step_size=5, gamma=0.1)
    print(kind, end=" ")
```

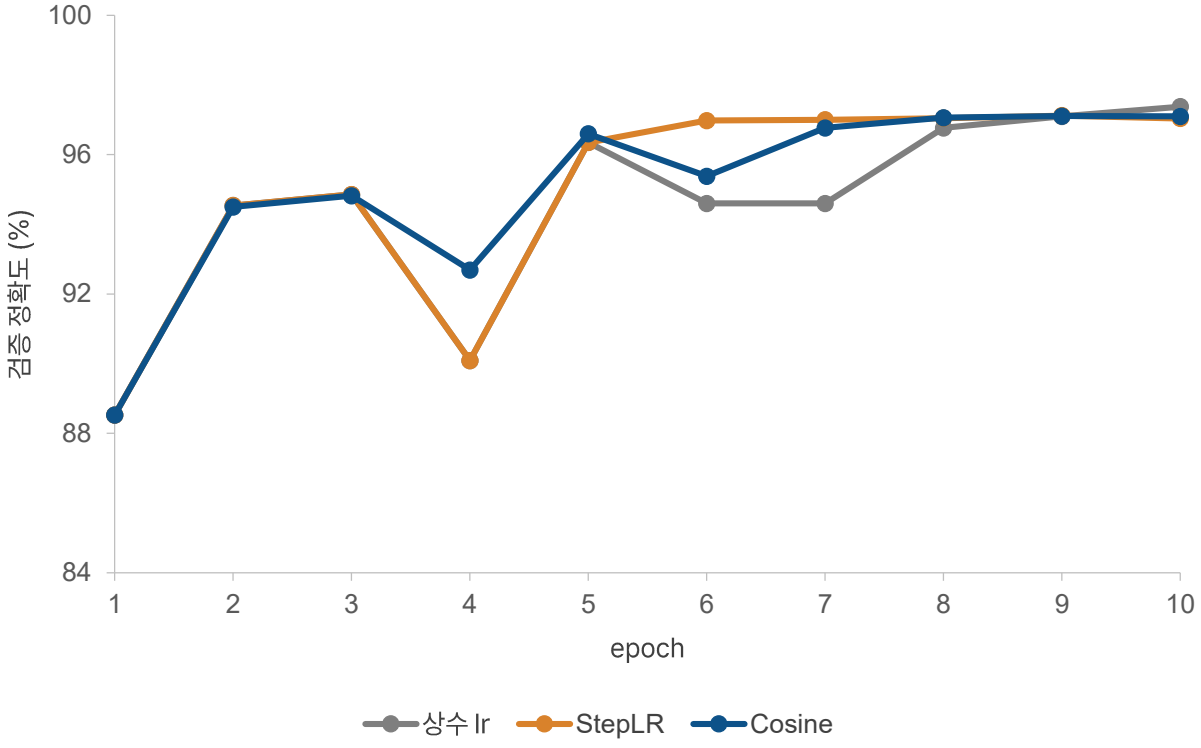
5. 실습 4: 정답 예시

실습 4. 정답 예시

```
sched_results = {}
for kind in ("const", "cos", "step"):      # 1. 방식 목록에 step을 더한다
    torch.manual_seed(0)
    model = CNN().to(device)
    opt = torch.optim.SGD(model.parameters(), lr=0.1)
    sched = None
    if kind == "cos":
        sched = torch.optim.lr_scheduler.CosineAnnealingLR(opt, T_max=10)
    if kind == "step":                     # 5 epoch마다 lr을 10분의 1로
        sched = torch.optim.lr_scheduler.StepLR(opt, step_size=5, gamma=0.1)
    print(kind, end=" ")
    sched_results[kind] = fit(model, lab_loader, 10, opt, sched)
# → (예) const best 10 97.38 last 97.38
# → (예) cos best 9 97.11 last 97.1
# → (예) step best 9 97.12 last 97.04
```

- 1. StepLR 학습
  - lr이 0.01로 줄어든 6 epoch 뒤로 안정되고 마지막 97.04%다 (곡선은 다음 장)
- 2. 비교
  - seed 0의 마지막 값은 상수 lr 97.38, Cosine 97.10, StepLR 97.04%로 비슷하다
- 3. lr 출력
  - step_size 3이면 0.1, 0.01, 0.001로 3 epoch마다 줄어든다

5. 결과: 끝이 안정된다



seed 0, 1만 징, SGD lr 0.1 실측

상수 lr	97.38	96.88	92.57
StepLR	97.04	97.22	97.38
Cosine	97.10	97.33	97.44

10 epoch 마지막 검증 정확도 (%), seed 0~2 실측

- 상수 lr은 seed에 따라 마지막 값이 크게 흔들린다
- scheduler는 끝을 안정시킨다, 끝 무렵 어느 epoch에서 멈춰도 비슷하다

5. 예제 문제, scheduler 진단

문제

1. StepLR(step_size=5, gamma=0.1)의 `sched.step()`을 학습 함수 안에서 배치마다 부르면 어떻게 될까?
2. CosineAnnealingLR(T_max=10)으로 20 epoch을 학습하면 11 epoch부터 lr은 어떻게 될까?
3. scheduler를 쓰면 정확도가 늘 오를까?

5. 예제 문제 풀이

1	lr이 금방 거의 0이 되어 학습이 멈춘다	1 epoch 뒤 lr 1e-32(거의 0), 검증 14.62%
2	0이 된 뒤 다시 커진다, T_max는 epoch 수와 맞춘다	11 epoch에 0, 20 epoch에 다시 0.098
3	아니다, 끝을 안정시키는 도구다	20 epoch, seed 세 번에서 어느 쪽도 늘 높지 않았다

- `sched.step()`은 **epoch마다 한 번**, `opt.step()` 다음에 부른다

PART 06

튜닝 절차와 실험 5

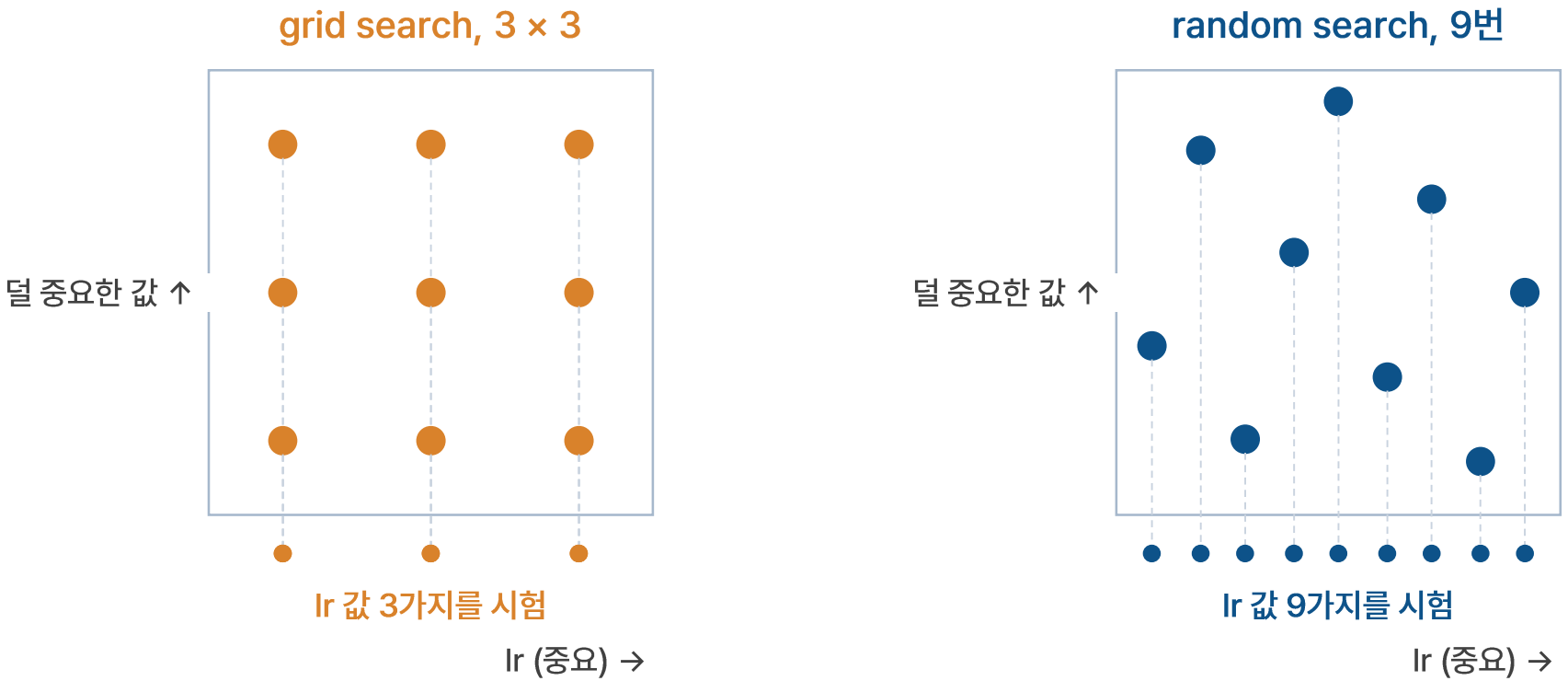
6. 튜닝 절차

- 작업 순서

1. 실제로 쓸 모델로 기준 설정의 검증 정확도를 기록한다
2. lr을 10배 간격으로 훑고, 좋은 구간을 촘촘히 본다
3. batch size, optimizer, weight decay를 정한다 (바꾸면 lr도 다시 본다)
4. scheduler로 끝을 안정시킨다
5. random search로 고르고, 상위 설정은 seed를 바꿔 다시 확인한다
6. 전체 데이터로 확인하고, test는 마지막에 한 번만 잰다

주의 고르는 것은 늘 검증 데이터로 한다, test로 고르면 test가 검증이 되어 버린다

6. grid search와 random search



lr 뽑기 $\eta = 10^u, \quad -4 \le u \le -2$

η는 lr, 지수 u를 −4~−2에서 고르게 뽑으면 lr은 0.0001~0.01에서 자릿수마다 고르게 뽑힌다,
u = −3이면 0.001

- 같은 횟수라면 random search가 **중요한 값**을 더 다양하게 시험한다

6. 예제 7-8: random search 8번

예제 7-8. CNNAll로 AdamW의 lr과 batch를 8번 뽑아 3 epoch씩

```
import random
rng = random.Random(0)                      # 뽑기용 seed, 누가 해도 같은 8개
trials = []
for t in range(8):
    lr = round(10 ** rng.uniform(-4, -2), 5)   # 0.0001 ~ 0.01, 자릿수마다 고르게
    bs = rng.choice([32, 64, 128, 256])        # 목록에서 하나를 뽑는다
    loader = DataLoader(lab_set, batch_size=bs, shuffle=True)
    torch.manual_seed(0)
    model = CNNAll().to(device)                # 지난시간 CNNAll, 실험 5의 모델
    opt = torch.optim.AdamW(model.parameters(), lr=lr)   # weight decay는 기본 0.01
    print(t + 1, "lr", lr, "batch", bs, end=" ")
    train_losses, val_losses, val_accs = fit(model, loader, 3, opt)   # 3 epoch만
    trials.append((round(max(val_accs), 2), lr, bs))
print(sorted(trials, reverse=True)[:2])          # 상위 두 개 (best가 큰 순서)
# → (예) [(97.19, 0.00147, 64), (97.05, 0.00488, 256)]
```

0.00147	64	97.19
0.00488	256	97.05
0.00172	128	97.02
0.00862	128	96.81
0.00102	128	96.43
0.00065	128	95.97
0.00019	32	95.70
0.00012	256	87.38

8번의 결과, 높은 순서 (%)

- 1등은 lr 0.00147, batch 64

6. 예제 7-9: 상위 두 설정을 seed 세 번으로

lr 0.00147, batch 64	97.19	97.62	97.18	97.33
lr 0.00488, batch 256	97.05	97.09	96.81	96.98

CNNAll, AdamW, 1만 장, 3 epoch, best 검증 정확도 (%) 실측

- 코드는 예제 7-8과 같은 모양, 바깥 for가 설정 두 개, 안쪽 for가 seed 세 개다
- seed 세 번 모두 1등 설정이 높았다, 이 설정을 **전체 데이터**로 가져간다
- 0.1%p 차이를 seed 하나로 판단하지 않는다, 흔들림보다 큰 차이인지 **다시 확인**한다

6. 실습 5: 나만의 random search (문제)

실습

1. 예제 7-8을 복사해 뽑기 seed만 `random.Random(1)`로 바꿔 8번 시험하기
2. 상위 두 설정의 lr이 예제 7-8과 비슷한 구간인지 비교하기
3. 뽑기 seed 0으로 weight decay도 [0.0, 0.01, 0.1] 중에서 함께 뽑아 보기

- 힌트
  - 2는 뽑기 seed 1의 상위 lr이 예제 7-8의 두 lr 사이에 드는지 본다
  - 3은 bs 줄 바로 아래에 `wd = rng.choice([0.0, 0.01, 0.1])`을 넣는다
  - 3의 opt 줄은 `opt = torch.optim.AdamW(model.parameters(), lr=lr, weight_decay=wd)`
  - 3의 print 줄은 `print(t + 1, "lr", lr, "batch", bs, "wd", wd, end=" ")`
  - 3의 trials 줄은 `trials.append((round(max(val_accs), 2), lr, bs, wd))`

6. 실습 5: 정답 예시

- 1. 뽑기 seed 1의 상위 두 개 (표의 오른쪽 칸)

1	lr 0.00147, batch 64, 97.19%	lr 0.00201, batch 64, 97.37%
2	lr 0.00488, batch 256, 97.05%	lr 0.00246, batch 128, 97.12%

- 2. 비교
  - 뽑기 seed 1의 상위 lr 두 개(0.00201, 0.00246)가 예제 7-8의 두 lr 사이(0.00147~0.00488)에 든다
- 3. weight decay도 뽑기
  - 1등은 lr 0.00924, wd 0.01(97.44%), 0.0092 근처도 더 확인할 후보다
  - 이번 8번은 lr과 batch에 따라 크게 갈렸다, weight decay만의 효과는 따로 재야 안다

6. 예제 7-10: 전체 데이터로 실험 5

예제 7-10. 실험 4와 같은 모델과 증강, 학습 조건만 튜닝한 값으로

```
torch.manual_seed(0)                          # 실험 4와 같은 seed, 같은 10 epoch
model = CNNAll().to(device)
opt = torch.optim.AdamW(model.parameters(), lr=0.00147)          # 튜닝으로 고른 값
sched = torch.optim.lr_scheduler.CosineAnnealingLR(opt, T_max=10)
best_acc, best_epoch = 0.0, 0
for epoch in range(10):
    train_one_epoch(model, train_aug_loader, opt)
    vl, va = evaluate(model, val_fast)
    sched.step()
    print(epoch + 1, round(va, 2))              # 진행 상황 보기
    if va > best_acc:
        best_acc, best_epoch = va, epoch + 1
        torch.save(model.state_dict(), "exp5.pt")   # 실험 5
print("best", best_epoch, round(best_acc, 2))   # → (예) best 10 98.98
```

- 결과
  - best 검증 정확도 98.98%(10 epoch), 실험 4는 98.77%였다
- 바꾼 것
  - 학습 조건만, AdamW lr 0.00147(weight decay 0.01), Cosine (batch 64는 그대로)
- 데이터
  - 앞 셀의 `train_aug_loader`, 지난 시간 증강을 넣은 전체 5만 장

6. 실험 기록표에 실험 5

Baseline	없음, MLP 784-100-10	97.25% (10)	97.34%	기준
실험 1	optimizer 설정, SGD lr 0.1 → Adam lr 0.001	97.43% (10)	97.59%	+0.18%p
실험 2	모델, MLP → CNN	98.54% (9)	98.67%	+1.29%p
실험 4	모델 CNNAll(BN, Dropout), 증강	98.77% (10)	98.88%	+1.52%p
실험 5	학습 조건, AdamW lr 0.00147(weight decay 0.01), Cosine	**98.98% (10)**	**99.04%**	**+1.73%p**

- seed 세 번으로 다시 재 보면 실험 5는 98.96~99.00%, 실험 4는 98.62~98.77%다
- 모델과 데이터는 그대로 두고 **학습 조건만** 바꿔 얻은 차이다

6. 예제 문제, 튜닝 진단

문제

1. 실습 5의 3에서 1등 lr은 0.00924로, 뽑기 범위(0.0001~0.01)의 끝 가까이였다, 다음에 할 일은?
2. SGD momentum으로 CNN 실험실에서 고른 1등을 실험 5 조건(CNNAll, 증강, 전체 데이터)으로 옮기면 늘 좋을까?
3. 실험 5는 실험 4보다 0.21%p 높았다, 학습 조건 중 무엇 덕분인지 말할 수 있을까?

6. 예제 문제 풀이

1	범위를 넓혀 확인한다, 범위 밖에 더 좋은 값이 있을 수 있다	같은 조건(batch 64, wd 0.01, 3 epoch)으로 넓혀 재 보니 lr 0.02는 96.64%, 0.05는 92.83%로 0.00924(97.44%)보다 낮았다
2	늘 그렇지는 않다, 마지막에 실제 조건 전체로 확인한다	SGD momentum으로 CNN 실험실에서 고른 1등(lr 0.0563, batch 64)이 실험 5 조건에서는 10.99%로 무너졌다
3	말할 수 없다, 여러 개가 함께 바뀌었다	optimizer, lr, weight decay, scheduler를 함께 바꿨다

- 튜닝은 실제 모델로 작게 고르고, 크게 확인하고, 마지막에 test의 순서다

6. 정리, 다음 시간

lr	한 번에 얼마나 옮길지	10배 간격으로 훑고 좋은 구간을 촘촘히
batch size	한 번에 몇 장을 보고 고칠지	바꾸면 lr도 함께 본다
epoch 수	몇 번 되풀이할지	검증 곡선이 평평해지는 곳, Early Stopping
optimizer	기울기로 고치는 방법	SGD 0.1, momentum 0.01, Adam 0.001에서 출발
weight decay	weight를 매번 조금씩 줄인다	Adam이면 AdamW로
scheduler	epoch에 따라 lr을 줄인다	T_max는 epoch 수와 맞춘다
튜닝 절차	작게 고르고 크게 확인한다	실제 모델로 random search, 상위는 seed 세 번

고르는 것은 검증으로, test는 마지막에 한 번

- 다음 시간에는 층 수와 채널 수 같은 구조 값을 실험으로 고르는 모델 구조 튜닝을 다룬다